WIA 1002 DATA STRUCTURE

SEM 2, SESSION 2025/2026

Dr. Zainab Malik
zainabmalik@um.edu.my
K1 (OCC 1 – 6)

Dr. Uzair Ishtiaq
uzair@um.edu.my
K2 (OCC 7 – 12)

Ts. Dr. Siti Nurliana Binti Jamalai@Jamali
siti.nurliana@um.edu.my
K3 (OCC 13 – 15)

ADT & Bag

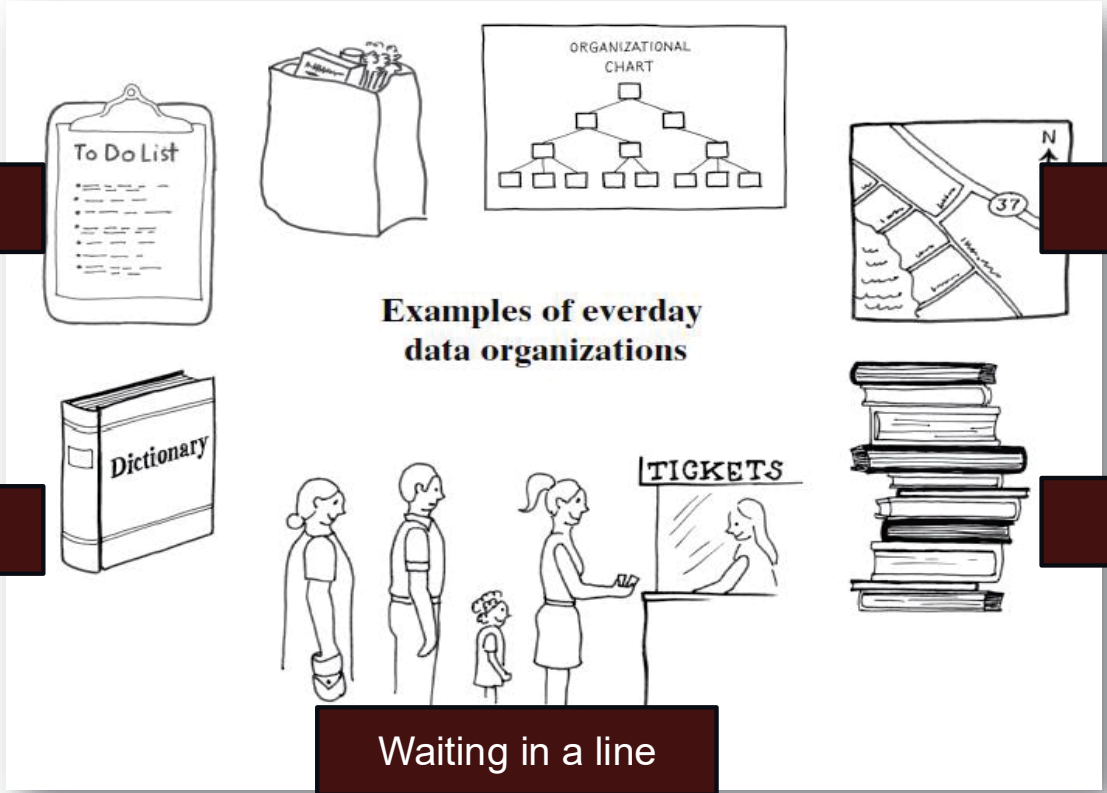
- Part 1: Abstract Data Type (ADT)
- Part 2: Bag

Abstract Data Type (ADT)

Data Organization

To-Do list

Dictionary



Waiting in a line

Road map

Stack of books

© 2016 Pearson Education, Ltd. All rights reserved.

Computer Data Organization

Just like real-world objects such as **lists, stacks, and dictionaries** help organize data efficiently, these concepts are represented in programming through **Abstract Data Types (ADTs)**.

© 2026 Pearson Education, Ltd. All rights reserved.

Collection & Container

- A ***collection*** is a general term for an **Abstract Data Type (ADT)** that **stores a group of objects**. It defines **what** operations can be performed on the group, such as adding, removing, or searching for elements.
- A ***container*** is a **class that implements** a collection. It provides the actual **implementation** of the collection in a specific programming language.

© 2026 Pearson Education, Ltd. All rights reserved.

Abstract Data Type

- An **ADT** is a **conceptual model** that defines:
  1. **The type of data** it stores.
  2. **The operations** that can be performed on the data.
- However, an ADT **does not specify how** the data is stored or **how** the operations are implemented.
- It is independent of any programming language and focuses on **what** should be done rather than **how** it is done.

© 2026 Pearson Education, Ltd. All rights reserved.

Abstract Data Type

- A data type that defines
  - **What** operations can be performed and what data it stores (**public to client)**
  - **How** the operations are implemented is **private (private to the client, hidden)**
- Focuses on data items and associated operations, but not implementation. As such, users interact only with the **interface**, not the internal logic.
- *Abstract* = irrelevant details are ignored, such as storage method.

Data Structure

A **data structure** is the actual **implementation** of an ADT in a programming language. It defines **how** data is stored and **how** operations are performed.

© 2026 Pearson Education, Ltd. All rights reserved.

ADT & Data Structure

ADT
List ADT → Supports adding, removing, and accessing elements.
Stack ADT → Follows Last-In-First-Out (LIFO) operations (push, pop).
Queue ADT → Follows First-In-First-Out (FIFO) operations (enqueue, dequeue).

implementation →

Data Structure
Array or Linkedlist

© 2026 Pearson Education, Ltd. All rights reserved.

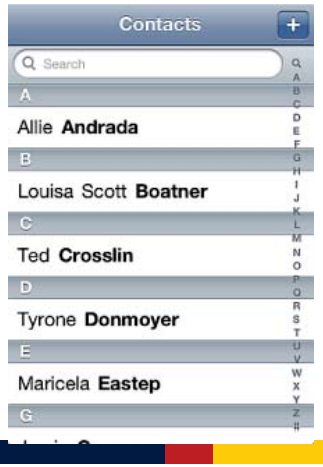
Example (Cinema Reservation System)

- Data?
  - Seats
  - Seats reserved or available
- Operations ?
  - Determine availability of seats
  - Reserve a seat
  - Cancel a reservation
  - Find a block of available seats
- Implementation?
  - Ignored

Example (Playing cards)

- The following are unrelated items :
- A deck of playing cards
- A set of index cards containing birthday information
- Telephone numbers stored in your cellular phone

- **What do they share in common?**



- Each one is a ***collection*** of elements.
- There is a first element.
- There is a second element, third element, and so on.
- There is a last element.
- Given an element other than the last element, there is a ‘‘next’’ element.
- Given an element other than the first element, there is a ‘‘previous’’ element.
- An element can be removed from the collection.
- An element can be added to the collection.
- A specified element can be located in the ***collection*** by systematically going through the ***collection***.

Advantage of ADTs

- Shares data and operations.
- Information hiding. ADT hides the implementation details of the operations and the data from the users of the ADT. Users can use the operations of an ADT without knowing how the operation is implemented

Bag

What items?

What operations?

Yes or No?

- Should the items be stored in a specific order?
- Can you keep repetitive items in the same bag?
- Is there a standard limit/number of items to be stored in the bag?

The ADT Bag

- Definition
  - A finite collection of objects in no particular order
  - Can contain duplicate items
- Possible behaviors
  - Add objects
  - Remove objects
  - Get number of items
  - Check for empty

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Class-Responsibility-Collaboration Card

Bag
Responsibilities
Get the number of items currently in the bag
See whether the bag is empty
Add a given object to the bag
Remove an unspecified object from the bag
Remove an occurrence of a particular object from the bag, if possible
Remove all objects from the bag
Count the number of times a certain object occurs in the bag
Test whether the bag contains a particular object
Look at all objects that are in the bag
Collaborations
The class of objects that the bag can contain

FIGURE 1-1 A CRC card for a class **Bag**

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Specifying a Bag

- Describe its data and specify in detail the methods that correspond to the bag's behaviors.
- Name the methods, choose their parameters, decide their return types, and write comments to fully describe their effect on the bag's data.

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

UML Notation

```
Bag
+getCurrentSize(): integer
+isEmpty(): boolean
+add(newEntry: T): boolean
+remove(): T
+remove(anEntry: T): boolean
+clear(): void
+getFrequencyOf(anEntry: T): integer
+contains(anEntry: T): boolean
+toArray(): T[]
```

FIGURE 1-2 UML notation for the class **Bag**

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

UML Notation

Pseudocode	UML Notation	Description
`getCurrentSize()`	`+getCurrentSize(): integer`	Reports the current number of objects in the bag. **Input:** None. **Output:** The number of objects currently in the bag.
`isEmpty()`	`+isEmpty(): boolean`	Checks whether the bag is empty. **Input:** None. **Output:** true if empty, false otherwise.
`add(newEntry)`	`+add(newEntry: T): boolean`	Adds a given object to the bag. **Input:** newEntry (an object). **Output:** true if added successfully, false otherwise.
`remove()`	`+remove(): T`	Removes an unspecified object from the bag, if possible. **Input:** None. **Output:** The removed object if successful, otherwise null.
`remove(anEntry)`	`+remove(anEntry: T): boolean`	Removes a specific object from the bag, if possible. **Input:** anEntry (an object). **Output:** true if removed successfully, false otherwise.
`clear()`	`+clear(): void`	Removes all objects from the bag. **Input:** None. **Output:** None.
`getFrequencyOf(anEntry)`	`+getFrequencyOf(anEntry: T): integer`	Counts how many times an object appears in the bag. **Input:** anEntry (an object). **Output:** The number of times anEntry occurs in the bag.
`contains(anEntry)`	`+contains(anEntry: T): boolean`	Checks whether a specific object exists in the bag. **Input:** anEntry (an object). **Output:** true if found, false otherwise.
`toArray()`	`+toArray(): T[]`	Retrieves all objects currently in the bag. **Input:** None. **Output:** A new array containing all elements in the bag.

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Design Decision

What to do for unusual conditions?

- Assume it won't happen
- Ignore invalid situations
- Guess at the client's intention
- Return value that signals a problem
- Return a boolean
- Throw an exception

© 2015 Pearson Education, Inc. Upper Saddle River, NJ. All rights reserved.

Interface

- can write Java headers for the bag's methods and organize them into a Java *interface* for the class that will implement the ADT.

```java
/**
   An interface that describes the operations of a bag of objects.
   @author Frank M. Carrano
*/
public interface BagInterface<T>
{
   /** Gets the current number of entries in this bag.
       @return  The integer number of entries currently in the bag. */
   public int getCurrentSize();

```

LISTING 1-1 A Java interface for a class of bags

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Interface

```java
/** Sees whether this bag is empty.
    @return  True if the bag is empty, or false if not. */
public boolean isEmpty();

/** Adds a new entry to this bag.
    @param newEntry  The object to be added as a new entry.
    @return  True if the addition is successful, or false if not. */
public boolean add(T newEntry);

/** Removes one unspecified entry from this bag, if possible.
    @return  Either the removed entry, if the removal
             was successful, or null. */
public T remove();

/** Removes one occurrence of a given entry from this bag, if possible.
    @param anEntry  The entry to be removed.
    @return  True if the removal was successful, or false if not. */
public boolean remove (T anEntry);

/** Removes all entries from this bag. */
```

LISTING 1-1 A Java interface for a class of bags

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Interface

```java
    /** Removes one occurrence of a given entry from this bag, if possible.
        @param anEntry  The entry to be removed.
        @return  True if the removal was successful, or false if not. */
    public boolean remove (T anEntry);

    /** Removes all entries from this bag. */
    public void clear();

    /** Counts the number of times a given entry appears in this bag.
        @param anEntry  The entry to be counted.
        @return  The number of times anEntry appears in the bag. */
    public int getFrequencyOf(T anEntry);

    /** Tests whether this bag contains a given entry.
        @param anEntry  The entry to locate.
        @return  True if the bag contains anEntry, or false if not. */
    public boolean contains(T anEntry);

    /** Retrieves all entries that are in this bag.
        @return  A newly allocated array of all the entries in the bag.
                 Note: If the bag is empty, the returned array is empty. */
    public T[] toArray();
} // end BagInterface
```

LISTING 1-1 A Java interface for a class of bags

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Implementing the ADT Bad

- Imagine we hire a programmer to implement the ADT bag in Java, given the interface and specifications that we have developed.
- We do not need to know *how* the programmer implemented the bag to be able to use it. We only need to know *what* the ADT bag does.

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Using the ADT Bag

- So, assume that we have a Java class, *Bag*, that implements the Java interface *BagInterface*
- Two examples on how we can use Bag: *OnlineShopper* and *PiggyBank*

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Example (Online Shopper)

```java
/**
    A class that maintains a shopping cart for an online store.
    @author Frank M. Carrano
*/
public class OnlineShopper
{
   public static void main(String[] args)
```

LISTING 1-2 A program that maintains a bag for online shopping

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Example (Online Shopper)

```
{
    Item[] items = {new Item("Bird feeder", 2050),
                    new Item("Squirrel guard", 1547),
                    new Item("Bird bath", 4499),
                    new Item("Sunflower seeds", 1295)};
    BagInterface<Item> shoppingCart = new Bag<>();
    int totalCost = 0;

    // Statements that add selected items to the shopping cart:
    for (int index = 0; index < items.length; index++)
    {
       Item nextItem = items[index]; // Simulate getting item from shopper
       shoppingCart.add(nextItem);
       totalCost = totalCost + nextItem.getPrice();
    } // end for

    // Simulate checkout
    while (!shoppingCart.isEmpty())
    {
       System.out.println(shoppingCart.remove());
```

LISTING 1-2 A program that maintains a bag for online shopping

© 2015 Pearson Education, Inc. Upper Saddle River, NJ. All rights reserved.

Example (Online Shopper)

```
        // Simulate checkout
        while (!shoppingCart.isEmpty())
           System.out.println(shoppingCart.remove());

        System.out.println("Total cost: " + "\t$" + totalCost / 100 + "." +
                           totalCost % 100);
    } // end main
} // end OnlineShopper
```

Output

```
Sunflower seeds $12.95
Bird bath       $44.99
Squirrel guard  $15.47
Bird feeder     $20.50
Total cost:     $93.91
```

LISTING 1-2 A program that maintains a bag for online shopping

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Example (Piggy Bank)

```java
/**
   A class that implements a piggy bank by using a bag.
   @author Frank M. Carrano
*/
public class PiggyBank
{
   private BagInterface<Coin> coins;

   public PiggyBank()
   {
      coins = new Bag<>();
   } // end default constructor

   public boolean add(Coin aCoin)
   {
      return coins.add(aCoin);
   } // end add
```

LISTING 1-3 A class of piggy banks

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Example (Piggy Bank)

```
14      public boolean add(Coin aCoin)
15      {
16         return coins.add(aCoin);
17      } // end add
18
19      public Coin remove()
20      {
21         return coins.remove();
22      } // end remove
23
24      public boolean isEmpty()
25      {
26         return coins.isEmpty();
27      } // end isEmpty
28   } // end PiggyBank
```

LISTING 1-3 A class of piggy banks

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Example (Piggy Bank)

```java
/**
   A class that demonstrates the class PiggyBank.
   @author Frank M. Carrano
*/
public class PiggyBankExample
{
   public static void main(String[] args)
   {
       PiggyBank myBank = new PiggyBank();

       addCoin(new Coin(1, 2010), myBank);
       addCoin(new Coin(5, 2011), myBank);
       addCoin(new Coin(10, 2000), myBank);
       addCoin(new Coin(25, 2012), myBank);

       System.out.println("Removing all the coins:");
       int amountRemoved = 0;

       while (!myBank.isEmpty())
       {
          Coin removedCoin = myBank.remove();
          System.out.println("Removed a " + removedCoin.getCoinName() + ".");
```

LISTING 1-4 A demonstration of the class **`PiggyBank`**

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Example (Piggy Bank)

```
19          while (!myBank.isEmpty())
20          {
21             Coin removedCoin = myBank.remove();
22             System.out.println("Removed a " + removedCoin.getCoinName() + ".");
23             amountRemoved = amountRemoved + removedCoin.getValue();
24          } // end while
25          System.out.println("All done. Removed " + amountRemoved + " cents.");
26       } // end main
27
28       private static void addCoin(Coin aCoin, PiggyBank aBank)
29       {
30          if (aBank.add(aCoin))
31             System.out.println("Added a " + aCoin.getCoinName() + ".");
32          else
33             System.out.println("Tried to add a " + aCoin.getCoinName() +
34                                ", but couldn't");
35       } // end addCoin
36    } // end PiggyBankExample
```

LISTING 1-4 A demonstration of the class **`PiggyBank`**

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Example (Piggy Bank)

Output

```
Added a PENNY.
Added a NICKEL.
Added a DIME.
Added a QUARTER.
Removing all the coins:
Removed a QUARTER.
Removed a DIME.
Removed a NICKEL.
Removed a PENNY.
All done. Removed 41 cents.
```

LISTING 1-4 A demonstration of the class **`PiggyBank`**

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Using ADT is like Using a Vending Machine

FIGURE 1-3 A vending machine

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Using ADT is like Using a Vending Machine

Vending Machine	ADT Bag
Can perform only tasks machine's interface presents.	Can perform only tasks specific to ADT
You must understand these tasks	Must adhere to the specifications of the operations of ADT
Cannot access the inside of the machine	Cannot access data inside ADT without ADT operations.
You can use the machine even though you do not know what happens inside.	Use the ADT, even if don't know how data is stored
Usable even with new insides.	Usable even with new implementation.

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Java Class Library: The Interface Set

```java
/** An interface that describes the operations of a set of objects. */
public interface SetInterface<T>
{
   public int getCurrentSize();
   public boolean isEmpty();

   /** Adds a new entry to this set, avoiding duplicates.
       @param newEntry  The object to be added as a new entry.
       @return  True if the addition is successful, or
                false if the item already is in the set. */
   public boolean add(T newEntry);

   /** Removes a specific entry from this set, if possible.
       @param anEntry  The entry to be removed.
       @return  True if the removal was successful, or false if not. */
   public boolean remove(T anEntry);
```

Listing 1-5 A Java interface for a class of sets

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Java Class Library: The Interface Set

```
13      /** Removes a specific entry from this set, if possible.
14          @param anEntry  The entry to be removed.
15          @return  True if the removal was successful, or false if not. */
16      public boolean remove(T anEntry);
17
18      public T remove();
19      public void clear();
20      public boolean contains(T anEntry);
21      public T[] toArray();
22  } // end SetInterface
```

Listing 1-5 A Java interface for a class of sets

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Additional Slides

- Bag implemented using array (ArrayBag)
- Bag implemented using list (LinkedBag)

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Fixed-Size Array (ArrayBag)

```
ArrayBag
-bag: T[]
-numberOfEntries: integer
-DEFAULT_CAPACITY: integer
+getCurrentSize(): integer
+isEmpty(): boolean
+add(newEntry: T): boolean
+remove(): T
+remove(anEntry: T): boolean
+clear(): void
+getFrequencyOf(anEntry: T): integer
+contains(anEntry: T): boolean
+toArray(): T[]
-isArrayFull(): boolean
```

FIGURE 2-2 UML notation for the class **`ArrayBag`**, including the class's data fields

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Fixed-Size Array (ArrayBag)

```java
/**
   A class of bags whose entries are stored in a fixed-size array.
   @author Frank M. Carrano
*/
public final class ArrayBag<T> implements BagInterface<T>
{
   private final T[] bag;
   private int numberOfEntries;
   private static final int DEFAULT_CAPACITY = 25;

   /** Creates an empty bag whose initial capacity is 25. */
   public ArrayBag()
   {
       this(DEFAULT_CAPACITY);
   } // end default constructor

   /** Creates an empty bag having a given initial capacity.
       @param capacity  The integer capacity desired. */
   public ArrayBag(int capacity)
```

LISTING 2-1 An outline of the class **`ArrayBag`**

Note : When a class header includes an implements clause, the class must define all of the methods in the interface.

© 2017 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Fixed-Size Array (ArrayBag)

```java
/** Creates an empty bag having a given initial capacity.
    @param capacity  The integer capacity desired. */
public ArrayBag(int capacity)
{
    // The cast is safe because the new array contains null entries.
    @SuppressWarnings("unchecked")
    T[] tempBag = (T[])new Object[capacity]; // Unchecked cast
    bag = tempBag;
    numberOfEntries = 0;
} // end constructor

/** Adds a new entry to this bag.
    @param newEntry  The object to be added as a new entry.
    @return  True if the addition is successful, or false if not. */
public boolean add(T newEntry)
{
    < Body to be defined >
} // end add

/** Retrieves all entries that are in this bag.
```

LISTING 2-1 An outline of the class **`ArrayBag`**

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

Fixed-Size Array (ArrayBag)

```
    /** Retrieves all entries that are in this bag.
        @return  A newly allocated array of all the entries in the bag. */
    public T[] toArray()
    {
        < Body to be defined >
    } // end toArray

    // Returns true if the arraybag is full, or false if not.
    private boolean isArrayFull()
    {
        < Body to be defined >
    } // end isArrayFull

    < Similar partial definitions are here for the remaining methods
      declared in BagInterface. >

    . . .
} // end ArrayBag
```

LISTING 2-1 An outline of the class **`ArrayBag`**

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

An Outline of the Class `LinkedBag`

```
/**
    A class of bags whose entries are stored in a chain of linked nodes.
    The bag is never full.
    @author Frank M. Carrano
*/
public final class LinkedBag<T> implements BagInterface<T>
{
   private Node firstNode;       // Reference to first node
   private int  numberOfEntries;

   public LinkedBag()
   {
      firstNode = null;
      numberOfEntries = 0;
   } // end default constructor

   // Implementations of the public methods declared in BagInterface go here...
```

LISTING 3-2 An outline of the class `LinkedBag`

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

An Outline of the Class LinkedBag

```
14        numberOfEntries = 0;
15    } // end default constructor
16
17    < Implementations of the public methods declared in BagInterface go here. >
18
19    . . .
20
21    private class Node // Private inner class
22    {
23        < See Listing 3-1. >
24    } // end Node
25 } // end LinkedBag
```

LISTING 3-2 An outline of the class **LinkedBag**

© 2015 Pearson Education, Inc., Upper Saddle River, NJ. All rights reserved.

References

1. Introduction Chapter and Chapter 1: *Data Structures and Abstractions with Java, 4e,* Frank Carrano
2. https://www.youtube.com/watch?v=HcxqzYsiJ3k
3. http://www.radford.edu/~nokie/classes/320/Abstract_Data_Types.html